

Data Management & Compute

CSS Bootcamp — Day 2

Émilien Schultz — emilien.schultz@ensae.fr

2026-09-17

Forewords

Who am I

▶ **Émilien Schultz**

- ▶ Research engineer / data scientist at CREST
- ▶ Research facilitator: I help on data & code
- ▶ *emilien.schultz@ensae.fr* — office 4044



Today's rule

Everything we do today has to survive **the loss of your computer**. Scientific computation needs robustness, not brittleness.

3 questions

- ▶ What is your background ?
- ▶ What is the **largest dataset** you ever opened?
- ▶ Did you ever meet limit to run scripts/computation?

Programme

1. Some elements of scientific computing
2. Reproducibility, open science & DevOps
3. Organising a data project
4. Storage vs compute: sizes, HPC
5. Onyxia & remote services

The philosophy of the day

The **unholy trinity**

- ▶ Computational service (**compute**)
- ▶ Scripts / software / treatments (**code**)
- ▶ **Data**

Each aspect has specific tools/best practices.

And now, also AI

Is it a fourth dimension ?

- ▶ Models
- ▶ Workflow
- ▶ Tooling
- ▶ ...

 What does it change ?

It changes a lot of practices, but not the general philosophy or architecture we are going to see today.

Some elements of scientific computing

Computational social science?

- ▶ **New volumes** of data in fields that were not computational: text, web, admin data, digital traces
- ▶ **Higher expectations** of reproducibility and long-term storage (journals, funders, DMPs)
- ▶ **New tools** (LLMs, embeddings, large models) need GPUs and remote compute
- ▶ More and more **collaboration**: several people, several machines, several months

Need of remote computers/services

Where could you run your code?

- ▶ **Local computer**
 - ▶ limit of resources / not collaborative / messy
- ▶ **Cloud services** (Google Colab, Kaggle)
 - ▶ proprietary, limited, black box
- ▶ **Institutional services** (Onyxia datalab, Jean Zay, CREST server)
 - ▶ conditions of access, availability
 - ▶ but shared storage, big machines, GPUs

How to size resources? (later)

What changes when you leave your laptop

- ▶ Nothing is persistent by default
- ▶ No mouse, no GUI: **command line**
- ▶ Resources are requested, shared, and (sometimes) paid
- ▶ Your credentials become the problem
- ▶ Everything has to be **written down**: scripts, not clicks

This is what makes it reproducible.

Reproducibility, open science & DevOps

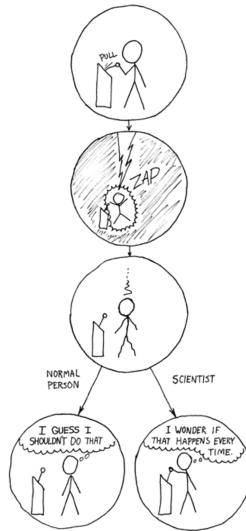
What is reproducibility

A result or project is never a one shot.

A result is reproducible when the same analysis steps performed on the same dataset consistently produce the same answer.

...on another machine, by another person, later.

Science loves reproducibility



Different flavours of reproducibility

		Data	
		Same	Different
Analysis	Same	Reproducible	Replicable
	Different	Robust	Generalisable

What it entails to be reproducible

Be able to walk the path again: connecting data, software and questions



Openness

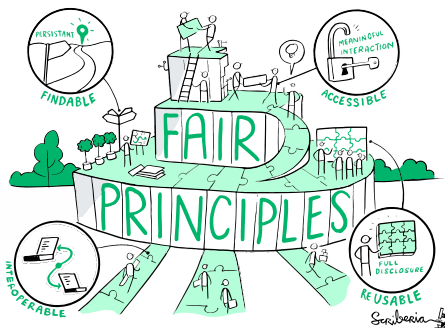
Both for data and code, create the possibility of reproducibility



Not always possible for data - especially in social sciences with sensitive informations

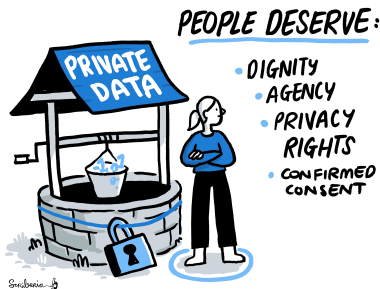
What is open science

- ▶ Clear methodologies
- ▶ Data accessibility: **FAIR** (Findable, Accessible, Interoperable, Reusable)
- ▶ Reusable software (open code, open formats)



How to conciliate privacy with reproducibility

- ▶ As open as possible, as closed as necessary
- ▶ Anonymise / pseudonymise
- ▶ Aggregate
- ▶ Synthetic data



How to achieve it: the minimal kit

1. A **Git repository** with a README.md: what, how to run, where the data is
2. A **pinned environment**: requirements.txt (or pyproject.toml), Python version
3. **Scripts**, not only notebooks
4. **Data separated from code**: raw data immutable, every transformation scripted
5. Random seeds, logged parameters, dated outputs

This is the deliverable of the afternoon.

Scripts, notebooks and packages

- ▶ Notebooks allow literate programming
 - ▶ very useful for exploration / prototyping
- ▶ Scripts are better for stability, automation, remote execution
- ▶ A pathway: notebook → script → package

Rule of thumb: if it runs more than once, it is a script.

The DevOps connection

DevOps (Software Development/Information technology operations)= practices that connect code and uses

- ▶ version control, automated tests, CI/CD
- ▶ containers, infrastructure as code
- ▶ logs, monitoring

MLOps = the same for data and models: data versioning, pipelines, experiment tracking ; **LLMOps**, you guess...

What research borrows from it

A new terminology that is invading computational practice

- ▶ **Containers** (Docker image): a frozen environment
- ▶ **Pipelines**: a Makefile / `run.sh` / snakemake with a clear order of steps
- ▶ **Logs**: know what happened when you were not looking
- ▶ **CI**: GitHub Actions running tests on each change
Research workflows should be designed with recomputation in mind

New vocabulary, new collaborations

You will work with research engineers, data engineers, sysadmins of the datalab / HPC.

Knowing their words (*pod, bucket, job, image, quota*) may allow new collaborations !

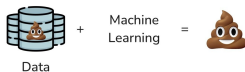
The reproducibility ladder

Level	What you have
0	a notebook on a laptop
1	script + README + Git
2	+ pinned environment + data on shared storage
3	+ container / CI
4	+ workflow manager on HPC

Organising a data project

Rule 0: know your data

- ▶ Have a look at your data
 - ▶ duplicates, empty data, encoding
- ▶ Before starting complex treatments
 - ▶ do simple treatments
 - ▶ ask yourself if the question and the data match



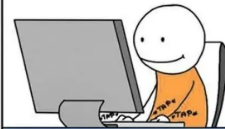
Rule 1: structuring a project

- ▶ **Separate code from data**
 - ▶ different logic, different tools, different sizes, different rights
- ▶ **Document / comment** your productions
 - ▶ be explicit (variables, README)
 - ▶ *yes, it is boring, but useful*
- ▶ **Modularise**: create distinct elements
 - ▶ one file per step
 - ▶ functions rather than repeated code

There will always be another Monday

UNFINISHED WORK

FRIDAY EVENING



PERFECT!
I'LL FINISH
THIS ON
MONDAY



MONDAY MORNING...



WHAT DOES
THIS MEAN!?!?

Rule 2: a common file system

```
project-name/
├── README.md           # what, how to run, where is the data
├── requirements.txt    # pinned environment
├── .gitignore          # data/, .env, outputs
├── data/              # NOT in git
│   ├── README.md      # -> where the data really is
│   ├── raw/           # original, immutable
│   ├── intermediate/
│   └── processed/     # analysis-ready
├── src/               # one script per step
├── docs/
└── reports/
    ├── figures/
    └── tables/
```

Rule 2 bis: paths

! Bad

```
df = pd.read_csv("/Users/emilien/Downloads/final2.csv")
```

💡 Good

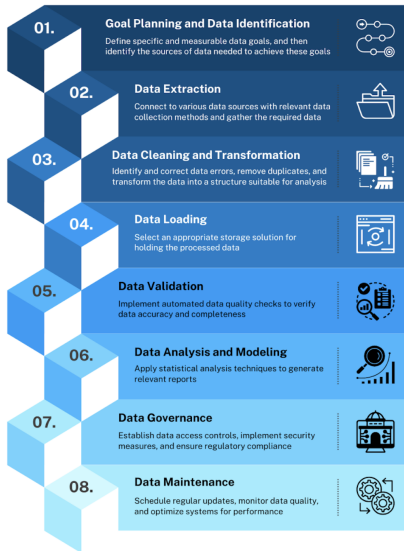
```
from pathlib import Path
import os
DATA = Path(os.environ.get("DATA_DIR", "data"))
df = pd.read_parquet(DATA / "raw" / "wiki.parquet")
```

Paths are configuration, not code.

Rule 3: organise the code for collaboration

- ▶ Comments in the script ; different files to modularize
- ▶ Git: one branch per person/feature, small commits, PR + review
- ▶ Log what you do (print is a start, logging is better)
- ▶ Issues as the shared to-do list
- ▶ Conventions > update the README

The data lifecycle : a specific philosophy



Where to store the data

During the project

- ▶ Laptop: temporary
- ▶ Institutional solutions
- ▶ Cloud solutions: with care (privacy)

After the project

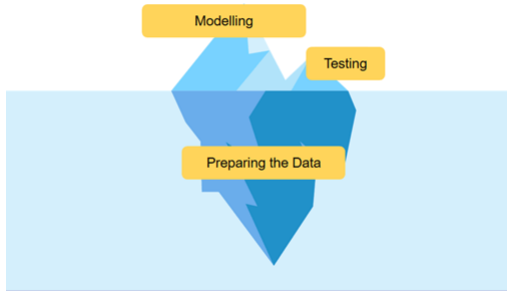
- ▶ Zenodo, Figshare, Hugging Face datasets, data.gouv.fr

Ideally, a Data Management Plan

Data management plan: a document that lists the data and how you handle it

- ▶ What types of data? What formats?
- ▶ What **volume**?
- ▶ Where stored **during**? Where **after**?
- ▶ Who can access it? Privacy?
- ▶ Naming and versioning strategy?

Rule 4: accept to spend time on data preparation



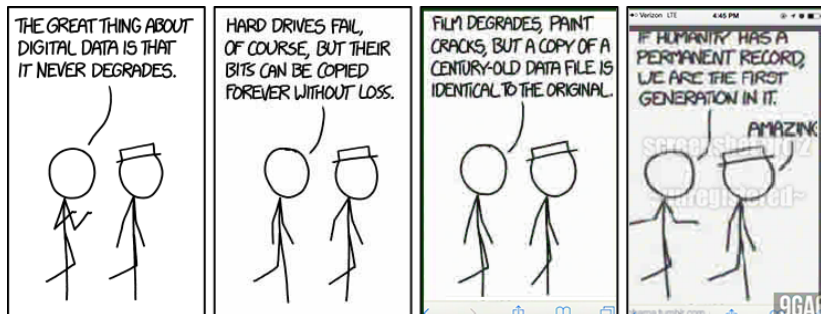
Avoid correcting your data manually

Build a workflow — write scripts

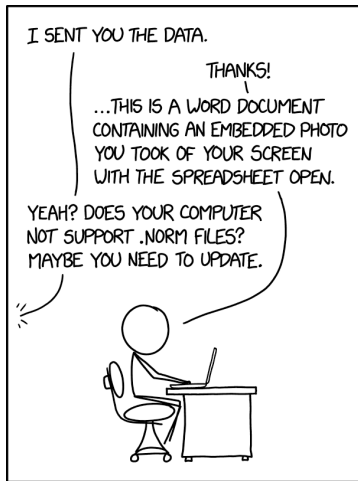


Let's talk about data format

Format is both information & affordance



What are the best data formats



SINCE EVERYONE SENDS STUFF THIS WAY ANYWAY, WE SHOULD JUST FORMALIZE IT AS A STANDARD.

Formats: a practical map

Format	Pros	Cons
CSV	universal, human-readable	no types, encoding & separators, slow, big
JSON / JSONL	nested, streams line by line	verbose
Parquet	typed, compressed, columnar, splittable	binary, needs a library
SQLite / DuckDB	relational, joins, SQL	one file, one writer

Suggestion: open format + think about how you will access it.

Data architecture: aim for tidy data

Take time to transform the data if needed

- ▶ Separate raw data from transformed data
- ▶ Move from unstructured to structured
- ▶ Tidy data

country	year	cases	population
Afghanistan	1999	1815	19997071
Afghanistan	2000	1866	20003360
Brazil	1999	30737	172006362
Brazil	2000	80488	174004898
China	1999	210258	1272015272
China	2000	210766	1280008583

variables

country	year	cases	population
Afghanistan	1999	1815	19997071
Afghanistan	2000	1866	20003360
Brazil	1999	30737	172006362
Brazil	2000	80488	174004898
China	1999	210258	1272015272
China	2000	210766	1280008583

observations

country	year	cases	population
Afghanistan	1999	1815	19997071
Afghanistan	2000	1866	20003360
Brazil	1999	30737	172006362
Brazil	2000	80488	174004898
China	1999	210258	1272015272
China	2000	210766	1280008583

values

Storage vs compute

What kind of computation do you need?

Resource	What it limits
CPU (cores)	speed, parallelism
RAM	the size of what you can hold: the true bottleneck for data
Disk	local = ephemeral & fast; object storage = persistent & shared
GPU	deep learning, embeddings, LLMs
Network	moving 17 GB is not free

How to think about sizes

Size	Example	Consequence
1 MB	a survey	anything works
100 MB	a year of admin data	Excel breaks, pandas is fine
1 GB	Several years of newspapers	think about RAM
10–100 GB	full Wikipedia, a month of Reddit	stream or sample, remote machine
1 TB+		you need a data engineer

Rules of thumb

- ▶ RAM needed $\approx 3-10\times$ the compressed parquet size (text)
- ▶ If it does not fit: **stream** (row groups, chunks, lazy engines) or **sample**
- ▶ Favour small data if you can



Otherwise

Solution 1: reduce your data

- ▶ work on a subset / sample
- ▶ read only the columns you need
- ▶ batch

Solution 2: parallelise

- ▶ more cores on one machine (polars, DuckDB, ...)
- ▶ more machines (Dask, Spark, HPC jobs)
- ▶ dedicated tools: xan for CSV

Where to run: a decision tree

- ▶ Fits in your laptop RAM, runs in minutes → **laptop**
- ▶ Needs more RAM / a GPU / hours → **Onyxia datalab**
- ▶ Needs days, many GPUs, thousands of jobs → **HPC** (Jean Zay, GENCI)

Start small, then scale

This process of scaling out the analysis can often be daunting, particularly for researchers new to computational methods

What is HPC

- ▶ A **cluster** of nodes with a shared file system
- ▶ A **job scheduler** (SLURM): you *submit* a script, you do not *run* it
- ▶ Queues, allocations, quotas, no GUI: CLI only
- ▶ Onyxia / Kubernetes is the middle ground: interactive, containerised, self-service

Know your Unix Commands



INSTITUT DE
RECHERCHES EN
INFORMATIQUE
SCIENTIFIQUE

IDRIS Cheat Sheet

Guide to Linux, module, SLURM & IDRIS commands

Accessing JEAN ZAY - HPE SGI 8600

Logging in from a terminal

ssh `login@jean-zay.idris.fr`

PutTY SSH client for Windows

Xming/XQuartzX11 X11 display server for Windows/Mac

Transferring files between JEAN ZAY and your system

sftp `login@jean-zay.idris.fr` destination

sftp> `put localSource JZDestination`

sftp> `get JZSource localDestination`

scp `localSource login@jean-zay.idris.fr:JZDestination`

scp `login@jean-zay.idris.fr:JZSource localDestination`

for large file (larger than 2GB) transfer

bbftp -e 'put `localSource JZDestination`' -u `login` -s `jean-zay.idris.fr`

bbftp -e 'get `JZSource localDestination`' -u `login` -s `jean-zay.idris.fr`

FileZilla SFTP client for Linux, Windows, Mac

WinSCP SFTP/SCP client for Windows

BBFTP In2p3 BBFTP client for Linux, Mac

Basic Linux file management

man `command` Display the manual page for command

pwd Print out the present working directory path

cd `dir` Change directory to `dir`

~ (tilde) Home directory

du -inodes -dl `dir` List inodes amount of directory `dir` and its subdirectories

find Find files in a directory

tree Display recursive directory listing

System information

hostname Show device/node name

date Return date time of system

top Display processor and memory usage

htop Display an augmented top viewer

watch `nvidia-smi` Provides monitoring information for GPUs

Viewing and editing text files

cat `file` Print entire content of file

more `file` Incrementally display content of file

less `file` Similar to more, but with additional features

head `file`/tail `file` Display file header / footer

emacs Extensible and customizable text editor

vim Minimalist yet powerful text editor

nano Simple text editor

Aliases and system variables

alias Automatically replace one command with another

env List all system variables

export `var=val` Create environment variable `$var` and set

The scale ladder

1. One file, on a sample → does the code work?
2. One file, full size → does it fit in memory? how long?
3. All files, one machine → parallelise
4. All files, many machines → HPC

Measure at each step: time, peak memory.

Note

A 32 GB pod left running for a week is not free. **Kill your services.**

Onyxia

What is Onyxia

- ▶ An open-source **datalab** platform developed by INSEE
- ▶ Instances:
 - ▶ GENES: <https://onyxia.lab.groupe-genes.fr>
 - ▶ INSEE SSPCloud: <https://datalab.sspcloud.fr>

User documentation

A common entry for different services

 **ENSAE-ENSAI Datalab**

[Rejoignez-nous sur Teams](#) [Documentation](#) [Gites](#) [Chat IA](#) [Connexion](#)

[Rédire](#)
[Accueil](#)
[Mon compte](#)
[Paramètres du projet](#)
[Catalogue de services](#)
[Mes services](#)
[Mes secrets](#)
[Explorateur de Fichiers](#)
[Explorateur de Données](#)



Bienvenue dans le Datalab

Onyxia de l'ENSAE-ENSAI

Travaillez avec Python ou R et disposez de la puissance dont vous avez besoin !





Un environnement ergonomique et des services à la demande

Analysez les données, faites du calcul distribué et profitez d'un large catalogue de services. Réservez la puissance de calcul dont vous avez besoin.

[Consulter le catalogue](#)



Une communauté active et enthousiaste à votre écoute

Profitez et partagez des ressources mises à votre disposition : tutoriels, formations et canaux d'échanges.

[Rejoindre la communauté](#)



Un espace de stockage de données rapide, flexible et en ligne

Pour accéder facilement à vos données et à celles mises à votre disposition depuis vos programmes - implémentation API S3

[Consulter des données](#)

What is a service

- ▶ A **container** (Docker image: OS + Python + Jupyter/VS Code/RStudio...)
- ▶ running in a **pod** on a cluster
- ▶ with the **resources you asked for** (CPU, RAM, GPU, disk)
- ▶ **ephemeral**: when it stops, its local disk is gone

! Important

The pod is compute, **not storage**.

How to launch one (live)

1. Catalogue → `vscode-python` or `jupyter-python`
2. Configure: resources, persistence, init script, Git, S3
3. Launch → wait → open
4. **My services:** see, delete, read the logs

Parameters that matter

- ▶ **Resources:** CPU request / limit, **RAM request / limit**, GPU
- ▶ **Persistence:** a volume (survives a restart of the service, not its deletion)
- ▶ **Init script:** install packages at start
- ▶ **Git:** a repository cloned at start, your name/email
- ▶ **S3:** credentials injected as environment variables



The **Kubernetes** tab shows the pod events. This is where OOMKilled appears.

How to access storage

- ▶ The **file browser** (“My files”): buckets, upload/download
- ▶ In the terminal: the `mc` client

```
mc ls s3/  
mc cp data/wiki.parquet s3/my-bucket/wiki/  
mc cp -r s3/my-bucket/wiki/ data/
```

- ▶ From Python: S3 is a URL + credentials in the environment

S3 from Python

```
import os
import pandas as pd

storage_options = {
    "client_kwargs": {"endpoint_url": os.environ["AWS_S3_E
    "key": os.environ["AWS_ACCESS_KEY_ID"],
    "secret": os.environ["AWS_SECRET_ACCESS_KEY"],
    "token": os.environ["AWS_SESSION_TOKEN"],
}
df = pd.read_parquet("s3://my-bucket/wiki/simple.parquet",
                     storage_options=storage_options)
```

Warning

Tokens **expire** ($\approx 24\text{h}$). Renew from *My account* $\rightarrow$ *Connect to storage*.

What survives what

Where	Service restarts	Service deleted	Datalab down
file on the pod	✗	✗	✗
persistent volume	✓	✗	✗
S3 bucket	✓	✓	✓ (backup)
code on GitHub	✓	✓	✓

This is the lesson of the afternoon, stated up front.

Secrets & security hygiene

Never commit

- ▶ S3 keys, Hugging Face tokens, GitHub tokens
- ▶ SSH private keys
- ▶ .env files, config with passwords
- ▶ Personal data

Use .gitignore + environment variables + a .env.example without values.

Tokens, passwords, keys

- ▶ **Password**: you, everywhere, forever → the worst
- ▶ **Token**: a service, a scope, an expiry → prefer read-only, short
- ▶ **Key pair**: a public half you share, a private half you never share

If you leak something: **revoke first**, then clean the history.

Prevention is much cheaper.

Hands on

The challenge

*Deploy a **reproducible** treatment of a **large dataset** on Onyxia, with a strict separation between **code** (GitHub), **storage** (S3) and **compute** (an ephemeral service). And keep it small :)*

Dataset: Wikipedia Monthly on Hugging Face

Have a look on the dataset

- ▶ Where is the data ?
- ▶ What is HuggingFace ?
- ▶ What is the format ?

What is a Parquet file

- ▶ **Columnar**: read only the columns you need
- ▶ **Typed** schema embedded in the file
- ▶ **Compressed** (snappy, zstd): 3–10× smaller than CSV
- ▶ Split in **row groups**: read a slice without loading everything
- ▶ The format of data lakes, Hugging Face, Spark, DuckDB, polars...

 On disk $\neq$ in memory

A 2 GB parquet of text can need **6–10 GB of RAM** once loaded in pandas.

The dataset

Subset | Files | Total | Notes |
20250702.en | 17 shards | 17.3 GB | shard 0 = **2.3 GB** |

Fields: id, url, title, text (clean plain text). Licence
CC-BY-SA.

Today: 20250702.en. Shards go from train_00016.parquet
(576 MB) up to train_00000.parquet (2.3 GB).

The steps

1. **Launch a service, update the common repository**
2. **A notebook**: download one shard, first analytics
3. **A script**: download the shards, keep the articles mentioning a keyword, upload the **built dataset** on S3
4. **Scale**: the same script on the big English shards, one file at a time, on a low-memory service
5. **Other tools** for the same job

! Rule of the afternoon

At some point I will ask you to delete your service and you will need to reset everything, so you need to keep track :)

Step 1 — A service and a common repository

Setup



Checklist

- ☐ One GitHub repository per group with a basic organization and a README.md
- ☐ A GitHub personal access token with write access to the group repository
- ☐ Each member launches a `vscode-python` service with **2 CPU / 4 GB RAM**, S3 enabled

Take your service in hand

- ▶ Clone the repository
- ▶ Have a look at the repository structure and the README skeleton
- ▶ Each member: add your name to the README in a new branch, commit, push, open a PR, merge one of your teammates' PR

Note

Check: does `git status` show your `data/` folder? It should not.

Step 2 — A notebook on one shard

Download one shard, in Python

Read the dataset card: fields, licence, shards. Then, in a notebook:

- ▶ See the HF CLI
- ▶ Find a way to download **the smallest shard**:
20250702.en/train_00016.parquet (576 MB) - Python or CLI
- ▶ Where did the file land? How big is it? (`ls -lh data/raw/...`)

Get a file

In Python

```
from huggingface_hub import hf_hub_download

path = hf_hub_download(
    repo_id="omarkamali/wikipedia-monthly",
    repo_type="dataset",
    filename="20250702.en/train_00016.parquet",
    local_dir="data/raw",
)
print(path)
```

In CLI

```
hf download omarkamali/wikipedia-monthly \
  20250702.en/train_00016.parquet \
  --repo-type dataset \
  --local-dir data/raw
```

First analytics

In a notebook, with pandas:

- ▶ load the shard, look at the columns and a few rows
- ▶ plot the histogram of text lengths → save it in reports/figures/
- ▶ print the title of the longest article
- ▶ extract the articles mentioning "data" → data/processed/

```
import pandas as pd
df = pd.read_parquet(path)
df["n"] = df["text"].str.len()
df["n"].plot.hist(bins=50, logy=True)
```

Update the repository

- ▶ The notebook goes to notebooks/, the figure to reports/figures/
- ▶ The data does **not** go to Git: check `git status`
- ▶ Commit, push, PR, review by a teammate, merge



Tip

Write in the README what the notebook does and how to get the shard.

Step 3 — A script: download, downsize, upload

From the notebook to a script

The notebook proved the idea on one shard. Now a **script** runs the whole pipeline:

`src/build_dataset.py`

1. takes a subset name, a list of shards and a keyword as arguments
2. **downloads** each shard from HF (local, data/raw/)
3. **downsizes**: keeps id, title, url, length (and text?) of the articles mentioning the keyword
4. **builds** one dataset from all the shards →
data/processed/wiki_<keyword>.parquet
5. **option** `--delete-raw`: deletes the raw shard once loaded (17 GB do not fit on the pod)

Raw shards are disposable. The built dataset is what matters.

Downsize

- ▶ How much smaller is the built dataset than the raw shards?
(`ls -lh`)
- ▶ What do you keep: the full text or only its length? What do you lose?
- ▶ Is it still useful to someone who did not see the raw data? → document it in `data/README.md`

Resources

- ▶ The Turing Way — Guide for Reproducible Research
- ▶ Onyxia user guide
- ▶ Stoudt et al. (2024). Ten simple rules for building and maintaining a responsible data science workflow. PLOS Comp Bio
- ▶ UCSD — Data management best practices
- ▶ Wickham (2014). Tidy data. JSS
- ▶ IDRIS cheat sheet (Jean Zay)
- ▶ Software Carpentry — The Unix Shell
- ▶ Perez-Riverol et al. (2016). Ten Simple Rules for Taking Advantage of Git and GitHub